Final Project Mata Kuliah PBKK
TodoList Backend & Frontend

API Documentation



Moh. Rizky Rahmadian Makkani

December 10, 2025

Contents

1 Overview 4
2 User Story 1: Membuat Task (CREATE) 4
2.1 Fitur 4
2.2 Backend Implementation 4
2.2.1 Endpoint 4
2.2.2 Request Body 4
2.2.3 Request Fields 4
2.2.4 Response (201 Created) 4
2.2.5 Service Layer 5
2.3 Frontend Implementation 5
2.3.1 Component: TodoForm 5
2.3.2 API Client 5
2.3.3 State Management 6
2.4 Testing 6
2.4.1 E2E Test 6
3 User Story 2: Melihat Tasks (READ) 7
3.1 Fitur 7
3.2 Backend Implementation 7
3.2.1 Endpoint 1: Get All Todos 7
3.2.2 Response (200 OK) 7
3.2.3 Endpoint 2: Get Single Todo 8
3.2.4 Service Implementation 8
3.3 Frontend Implementation 8
3.3.1 Components 8
3.3.2 Data Loading 8
4 User Story 3: Search & Filter Tasks 9
4.1 Fitur 9
4.2 Backend Implementation 10
4.2.1 Query Processing 10
4.3 Frontend Implementation 10
4.3.1 Components 10
4.3.2 Filter Flow 10
5 User Story 4: Mark Tasks Complete/Incomplete (UPDATE) 11
5.1 Fitur 11
5.2 Backend Implementation 11
5.2.1 Endpoint 11
5.2.2 Request Body (All Optional) 11
5.2.3 Response (200 OK) 12
5.2.4 Service Implementation 12
5.3 Frontend Implementation 12
5.3.1 Update Completion 12
5.3.2 Edit Todo Form 12
6 User Story 5: Edit atau Delete Tasks (UPDATE & DELETE) 13
6.1 Fitur 13
6.2 Backend Implementation 13
6.2.1 Delete Endpoint 13

6.2.2 Response (200 OK) 13
6.2.3 Delete Service 13
6.3 Frontend Implementation 13
6.3.1 Delete Flow 13
7 User Story 6: Organisasi Tasks dengan Categories 14
7.1 Fitur 14
7.2 Implementation 15
7.2.1 Create with Category 15
7.2.2 Update Category 15
7.2.3 Filter by Category 15
7.3 Frontend Implementation 15
7.3.1 Category Input 15
7.3.2 Category Filter 15
8 User Story 7: Lihat Tasks User Lain 16
8.1 Fitur 16
8.2 Backend Implementation 16
8.2.1 Search Users 16
8.2.2 Get User's Public Todos 16
8.2.3 Main List - Mixed Todos 16
8.3 Frontend Implementation 17
8.3.1 User Search Component 17
8.3.2 View Public Todos 17
9 User Story 8: Email Reminders 17
9.1 Fitur 17
9.2 Backend Implementation 18
9.2.1 Scheduler Service 18
9.2.2 Email Template 18
9.3 Configuration 18
9.4 Frontend Role 18

1 Overview

Pada Final Project ini peserta didik diminta untuk mengimplementasikan seluruh materi dalam mata kuliah Pemrograman Berbasis Kerangka Kerja dengan beberapa ketentuan yang telah dijelaskan dalam user story pada file soal Final Project.

2 User Story 1: Membuat Task (CREATE)

Sebagai user, saya ingin membuat task baru agar dapat melacak pekerjaan yang harus dilakukan.

2.1 Fitur

- Set judul task (wajib)
- Tambah deskripsi opsional
- Set prioritas (low, medium, high)
- Set tanggal deadline
- Kategorisasi task
- Set visibilitas (public/private)
- Lampirkan gambar
- Autentikasi (JWT Bearer token)

2.2 Backend Implementation

2.2.1 Endpoint

`POST /todos` (Autentikasi diperlukan)

2.2.2 Request Body

```
{
  "title": "Finish project report",
  "description": "Complete the quarterly financial report",
  "priority": "high",
  "dueDate": "2025-12-15T10:00:00.000Z",
  "category": "Work",
  "isPublic": false,
  "imagePath": "uploads/image-1234567890.jpg"
}
```

2.2.3 Request Fields

Field	Type	Required	Deskripsi
`title`	string	Ya	Judul task, min 1 char
`description`	string	Tidak	Deskripsi detail
`priority`	enum	Tidak	low/medium/high (default: medium)
`dueDate`	ISO 8601	Tidak	Format: 2025-12-15T10:00:00.000Z
`category`	string	Tidak	Custom category/tag
`isPublic`	boolean	Tidak	Default: false
`imagePath`	string	Tidak	Path dari `/upload` endpoint

2.2.4 Response (201 Created)

```
{
  "id": "uuid-string",
  "title": "Finish project report",
```

```
  "completed": false,
  "priority": "high",
  "category": "Work",
  "isPublic": false,
  "userId": "user-uuid",
  "user": {
    "id": "user-uuid",
    "username": "johndoe"
  },
  "createdAt": "2025-12-09T08:30:00.000Z",
  "updatedAt": "2025-12-09T08:30:00.000Z"
}
```

2.2.5 Service Layer

```
async create(createTodoDto: CreateTodoDto, userId: string) {
  const todo = await this.prisma.todo.create({
    data: {
      title: createTodoDto.title,
      description: createTodoDto.description,
      priority: createTodoDto.priority || 'medium',
      dueDate: createTodoDto.dueDate,
      category: createTodoDto.category,
      isPublic: createTodoDto.isPublic || false,
      imagePath: createTodoDto.imagePath,
      userId,
    },
    include: {
      user: { select: { id: true, username: true, email: true } },
    },
  })
  return todo
}
```

2.3 Frontend Implementation

2.3.1 Component: TodoForm

Flow:

1. User klik tombol “Buat Todo Baru”
2. Form modal terbuka dengan field kosong
3. User isi title (wajib), description, priority, category, due date, isPublic
4. Optional: upload gambar via `apiClient.uploadImage(file)`
5. User submit form
6. API call: `apiClient.createTodo(todo)`
7. Sukses: refresh list

2.3.2 API Client

```
async createTodo(todo: Omit<Todo, "id" | "createdAt" | "updatedAt" | "userId">) {
  return this.fetchWithAuth(`${API_URL}/todos`, {
    method: "POST",
    body: JSON.stringify(todo),
  })
}
```

2.3.3 State Management

```
const [formData, setFormData] = useState({
  title: "", description: "", priority: "medium",
  category: "", isPublic: false, imagePath: "",
})

const handleSubmit = async () => {
  try {
    await apiClient.createTodo(formData)
    toast({ title: "Todo berhasil dibuat" })
    await loadTodos()
  } catch (error) {
    toast({ title: "Error", description: error.message, variant: "destructive" })
  }
}
```

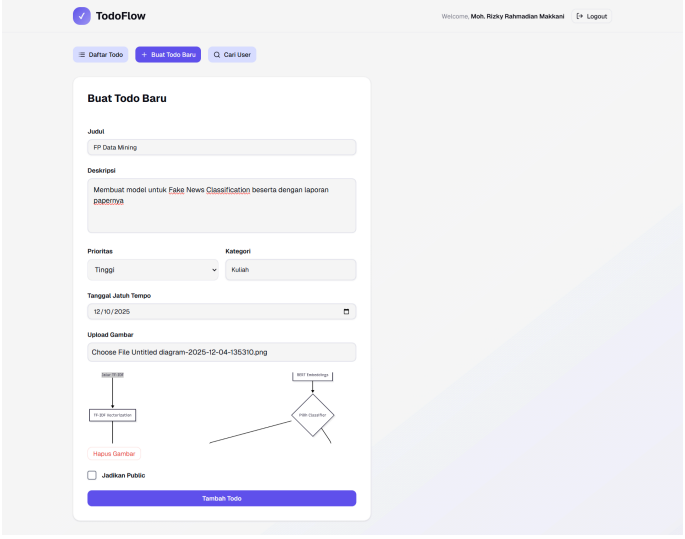
2.4 Testing

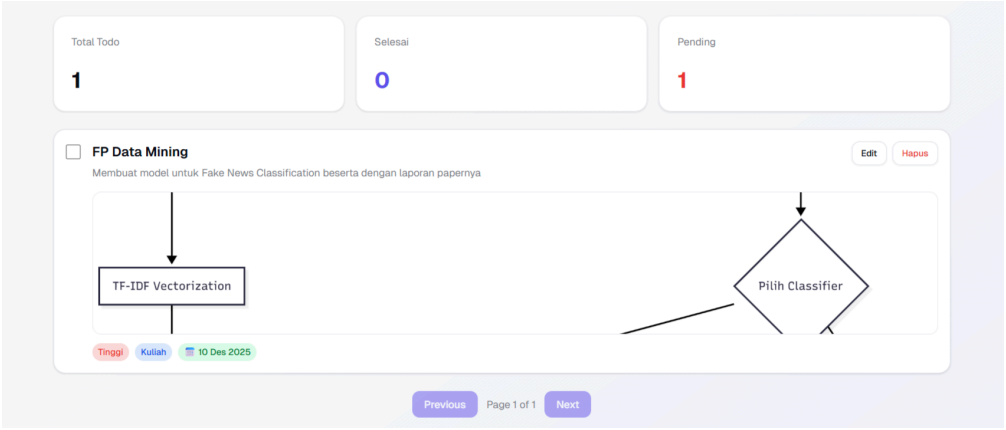
2.4.1 E2E Test

```
it('should create a todo successfully', async () => {
  const createTodoDto = {
    title: 'Create New Task',
    priority: 'high',
    category: 'Work',
  }

  const response = await request(app.getHttpServer())
    .post('/todos')
    .set('Authorization', `Bearer ${accessToken}`)
    .send(createTodoDto)
    .expect(201)

  expect(response.body).toHaveProperty('id')
  expect(response.body.title).toBe('Create New Task')
})
```





3 User Story 2: Melihat Tasks (READ)

Sebagai user, saya ingin melihat semua task dengan pagination dan detail task.

3.1 Fitur

- List semua task dengan pagination
- Filter berdasarkan status, prioritas, kategori, due date
- Search berdasarkan judul
- Lihat informasi detail untuk task tertentu
- Lihat task sendiri dan task public dari user lain
- Tampil statistik (completed/pending count)

3.2 Backend Implementation

3.2.1 Endpoint 1: Get All Todos

`GET /todos` (Autentikasi diperlukan)

Parameter	Type	Deskripsi
`search`	string	Cari berdasarkan title (partial match)
`priority`	enum	low/medium/high
`category`	string	Filter kategori
`isCompleted`	boolean	Filter status
`page`	number	Halaman (default: 1)
`limit`	number	Items per halaman (default: 10)

3.2.2 Response (200 OK)

```
{
  "data": [
    {
      "id": "todo-uuid",
      "title": "Task title",
      "completed": false,
      "priority": "high",
      "category": "Work",
      "isPublic": false,
      "createdAt": "2025-12-09T08:30:00.000Z"
    }
  ],
```

```
  "meta": {
    "total": 25,
    "page": 1,
    "lastPage": 3,
    "completedCount": 10,
    "pendingCount": 15
  }
}
```

3.2.3 Endpoint 2: Get Single Todo

`GET /todos/:id` (Autentikasi diperlukan)

3.2.4 Service Implementation

```
async findAll(userId: string, filter: TodoFilterDto) {
  const where = {
    OR: [
      { userId },
      { userId: { not: userId }, isPublic: true },
    ],
  }

  const [todos, total, completedCount, pendingCount] = await this.prisma.
$transaction([
    this.prisma.todo.findMany({
      where, skip: (filter.page - 1) * filter.limit,
      take: filter.limit, orderBy: { createdAt: 'desc' },
    }),
    this.prisma.todo.count({ where }),
    this.prisma.todo.count({ where: { ...where, completed: true } }),
    this.prisma.todo.count({ where: { ...where, completed: false } }),
  ])

  return {
    data: todos,
    meta: { total, page: filter.page, lastPage: Math.ceil(total / filter.limit),
      completedCount, pendingCount },
  }
}
```

3.3 Frontend Implementation

3.3.1 Components

TodoDashboard: Muat semua item di mount, kelola filter, paginasi.

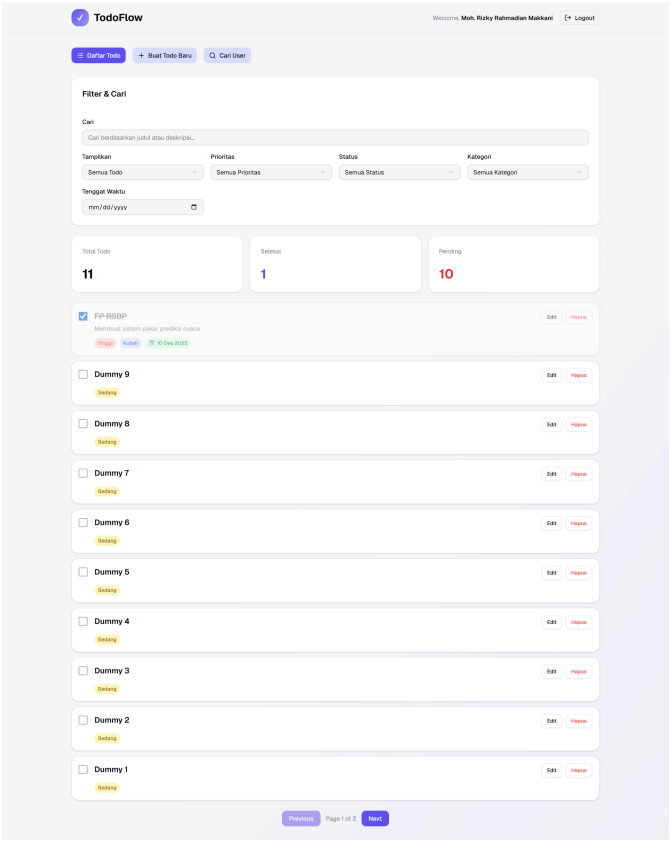
TodoList: Tampilkan item yang dipagination, handle posisi kosong.

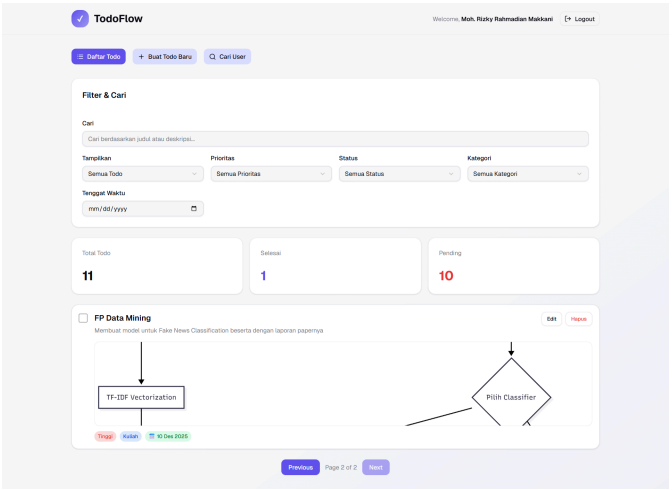
TodoItem: Card individu dengan judul, lencana prioritas, status, dan tanggal jatuh tempo.

3.3.2 Data Loading

```
const loadTodos = async (pageToLoad = 1) => {
  const response = await apiClient.getTodos({
    search: searchQuery,
    priority: filterPriority !== 'all' ? filterPriority : undefined,
    isCompleted: filterStatus !== 'all' ? filterStatus === 'completed' : undefined,
    page: pageToLoad, limit: 10,
  })
```

```
  setTodos(response.data)
  setCompletedCount(response.meta.completedCount)
  setPendingCount(response.meta.pendingCount)
}
```





4 User Story 3: Search & Filter Tasks

Sebagai user, saya ingin mencari dan filter task agar dapat menemukan task dengan cepat.

4.1 Fitur

- Search task berdasarkan judul (partial match)
- Filter berdasarkan prioritas (low, medium, high)
- Filter berdasarkan kategori

- Filter berdasarkan status (completed/pending)
- Filter berdasarkan due date
- Filter berdasarkan visibilitas (public/private)
- Kombinasi multiple filters
- Pagination untuk hasil search

4.2 Backend Implementation

4.2.1 Query Processing

```
async findAll(userId: string, filter: TodoFilterDto) {
  let where = { OR: [{ userId }, { userId: { not: userId }, isPublic: true }] }

  if (filter.search) {
    where['AND'] = { title: { contains: filter.search, mode: 'insensitive' } }
  }
  if (filter.priority) where['AND'] = { ...where['AND'], priority: filter.priority }
  if (filter.category) where['AND'] = { ...where['AND'], category: filter.category }
  if (filter.isCompleted !== undefined) {
    where['AND'] = { ...where['AND'], completed: filter.isCompleted }
  }
  if (filter.dueDate) {
    where['AND'] = { ...where['AND'], dueDate: { gte: new Date(filter.dueDate) } }
  }

  return this.prisma.todo.findMany({
    where, skip: (filter.page - 1) * filter.limit,
    take: filter.limit, orderBy: { createdAt: 'desc' },
  })
}
```

4.3 Frontend Implementation

4.3.1 Components

TodoFilters: Search input, priority/status/category dropdowns

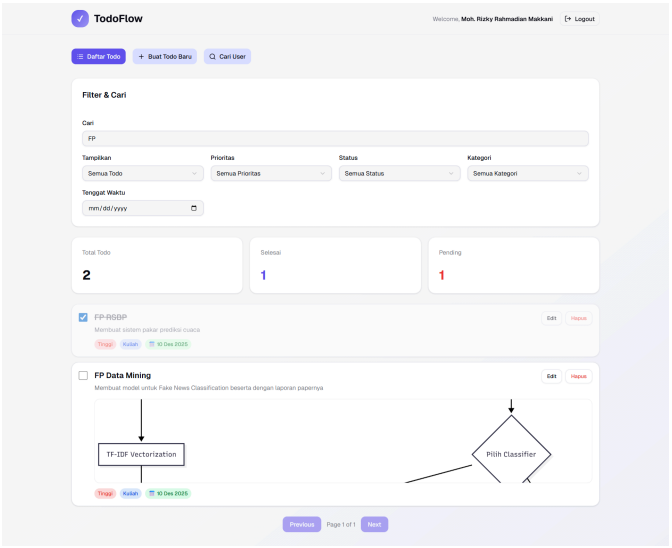
TodoDashboard: Listen to filter changes, reload todos

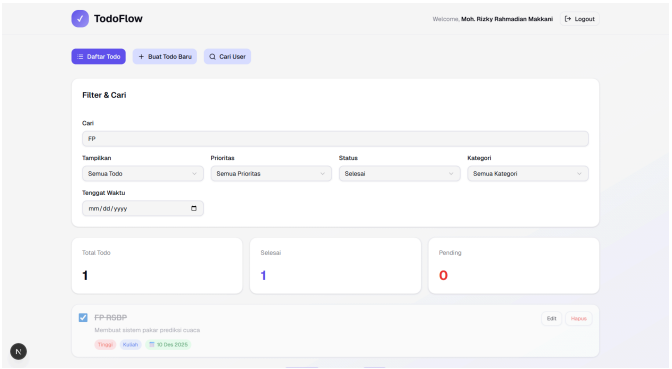
4.3.2 Filter Flow

```
const [searchQuery, setSearchQuery] = useState("")
const [filterPriority, setFilterPriority] = useState("all")
const [filterStatus, setFilterStatus] = useState("all")

// Debounced search
const handleSearch = (query) => {
  setSearchQuery(query)
}

// Reload on filter change
useEffect(() => {
  loadTodos(1)  // Reset to page 1
}, [searchQuery, filterPriority, filterStatus])
```





5 User Story 4: Mark Tasks Complete/Incomplete (UPDATE)

Sebagai user, saya ingin menandai task sebagai complete atau incomplete untuk melacak progress.

5.1 Fitur

- Toggle status completion task
- Update detail task (title, description, priority, dll.)
- Ubah due date, kategori, visibilitas
- Update gambar terlampir
- Hanya owner yang bisa update task mereka

5.2 Backend Implementation

5.2.1 Endpoint

`PATCH /todos/:id` (Autentikasi diperlukan)

5.2.2 Request Body (All Optional)

```
{
  "title": "Updated title",
  "description": "New description",
  "completed": true,
  "priority": "medium",
  "category": "Personal",
```

```
  "isPublic": true,
  "imagePath": "uploads/new-image.jpg",
  "dueDate": "2025-12-20T10:00:00.000Z"
}
```

5.2.3 Response (200 OK)

Returns updated todo object

5.2.4 Service Implementation

```
async update(id: string, userId: string, updateTodoDto: UpdateTodoDto) {
  const todo = await this.prisma.todo.findUnique({ where: { id } })

  if (todo.userId !== userId) {
    throw new ForbiddenException('Cannot update other users todos')
  }

  return this.prisma.todo.update({
    where: { id },
    data: updateTodoDto,
    include: { user: { select: { id: true, username: true } } },
  })
}
```

5.3 Frontend Implementation

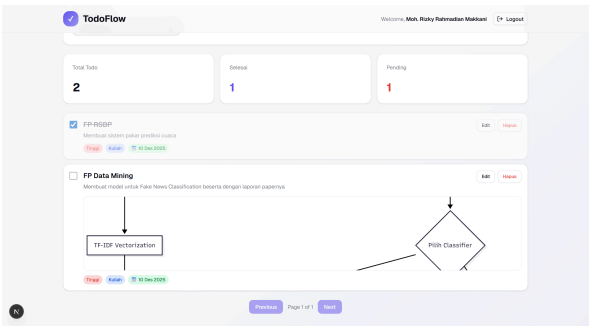
5.3.1 Update Completion

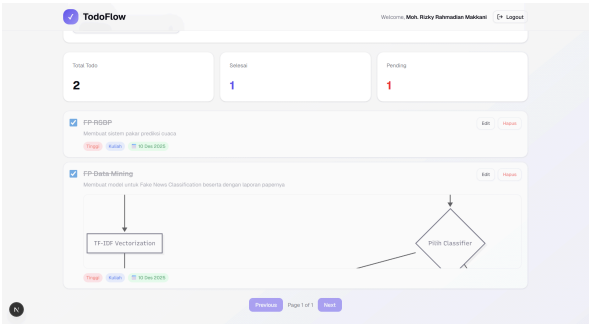
```
const toggleComplete = async (todoId: string, currentStatus: boolean) => {
  try {
    await apiClient.updateTodo(todoId, { completed: !currentStatus })
    await loadTodos()
    toast({ title: "Todo updated" })
  } catch (error) {
    toast({ title: "Error", variant: "destructive" })
  }
}
```

5.3.2 Edit Todo Form

Flow:

1. User klik Edit button pada todo
2. Form terbuka dengan data pre-filled
3. User memodifikasi fields yang ingin diubah
4. Submit: `apiClient.updateTodo(id, updates)`
5. Sukses: refresh list, close modal





6 User Story 5: Edit atau Delete Tasks (UPDATE & DELETE)

Sebagai user, saya ingin edit atau delete task untuk mengelola task list dengan efektif.

6.1 Fitur

- Edit field task apapun (partial update)
- Update beberapa field sekaligus
- Delete task secara permanen
- Authorization check (hanya owner)
- Cascade delete (hapus data terkait)
- Tidak bisa edit/delete task user lain

6.2 Backend Implementation

6.2.1 Delete Endpoint

`DELETE /todos/:id` (Autentikasi Diperlukan)

6.2.2 Response (200 OK)

```
{
  "message": "Todo deleted successfully",
  "id": "todo-uuid"
}
```

6.2.3 Delete Service

```
async delete(id: string, userId: string) {
  const todo = await this.prisma.todo.findUnique({ where: { id } })

  if (todo.userId !== userId) {
    throw new ForbiddenException('Cannot delete other users todos')
  }

  return this.prisma.todo.delete({ where: { id } })
}
```

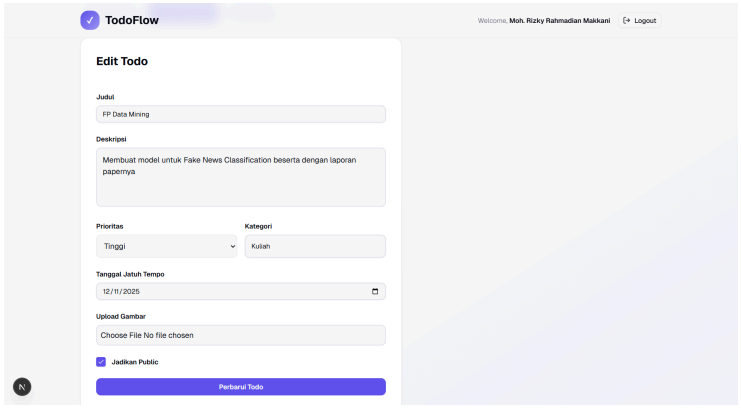
6.3 Frontend Implementation

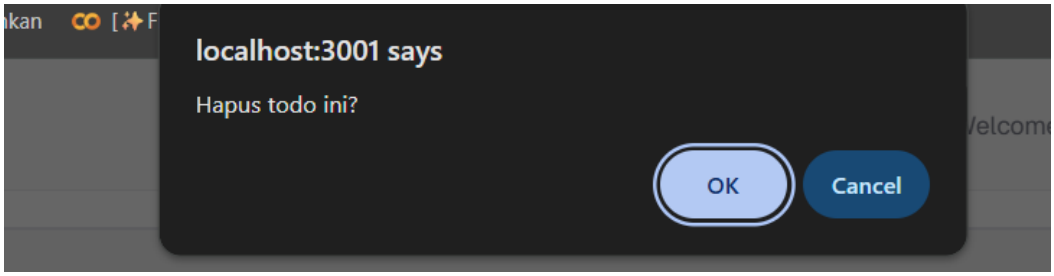
6.3.1 Delete Flow

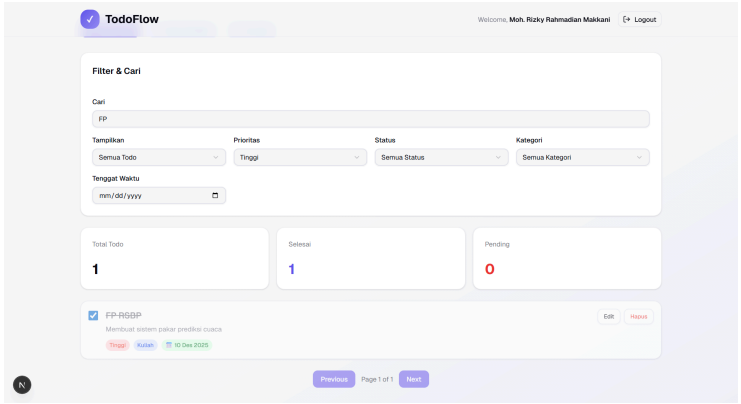
```
const handleDelete = async (todoId: string) => {
  const confirmed = confirm("Apakah Anda yakin ingin menghapus todo ini?")

  if (confirmed) {
```

```
    try {
      await apiClient.deleteTodo(todoId)
      setTodos(prev => prev.filter(t => t.id !== todoId))
      toast({ title: "Todo deleted" })
    } catch (error) {
      toast({ title: "Error", variant: "destructive" })
    }
  }
}
```







7 User Story 6: Organisasi Tasks dengan Categories

Sebagai user, saya ingin mengorganisir task ke dalam kategori atau tag untuk organisasi yang lebih baik.

7.1 Fitur

- Buat custom categories (free text)
- Assign kategori saat membuat task
- Update kategori task
- Filter task berdasarkan kategori

- Search lintas kategori
- Tidak ada predefined category list (flexible tagging)

7.2 Implementation

7.2.1 Create with Category

`POST /todos` dengan field `category`

```
{
  "title": "Project kickoff",
  "category": "Work"
}
```

7.2.2 Update Category

`PATCH /todos/:id` dengan field `category`

```
{
  "category": "Personal"
}
```

7.2.3 Filter by Category

```
GET /todos?category=Work
```

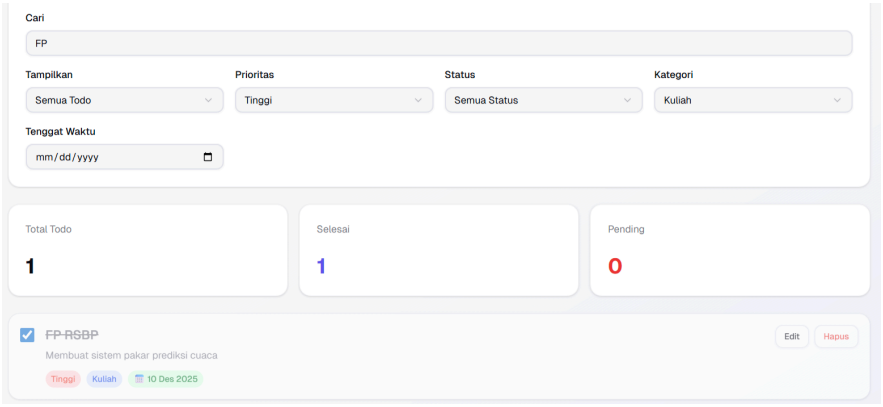
7.3 Frontend Implementation

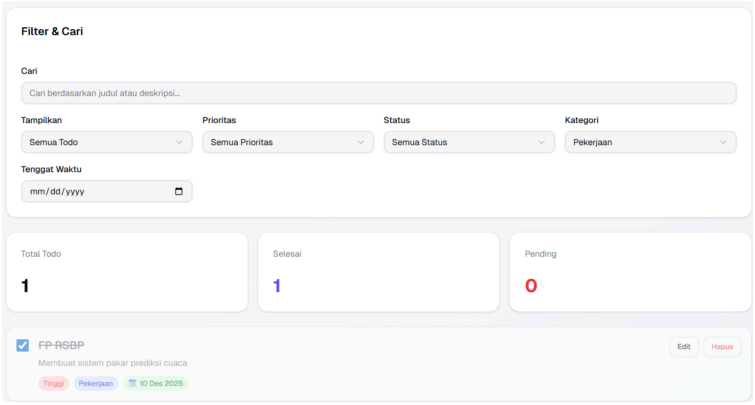
7.3.1 Category Input

```
<input
  type="text"
  placeholder="Category (e.g., Work, Personal, Shopping)"
  value={category}
  onChange={(e) => setCategory(e.target.value)}
/>
```

7.3.2 Category Filter

```
<select value={filterCategory} onChange={(e) => setFilterCategory(e.target.value)}>
  <option value="all">All Categories</option>
  <option value="Work">Work</option>
  <option value="Personal">Personal</option>
  {/* Dynamic from existing todos */}
</select>
```





8 User Story 7: Lihat Tasks User Lain

Sebagai user, saya ingin browse dan lihat todo list public dari user lain.

8.1 Fitur

- Search user berdasarkan username atau email
- Lihat profile user
- Akses task public dari user lain (read-only)
- Tidak bisa edit atau delete task user lain
- Task private tetap tersembunyi
- Authorization enforcement (403 untuk private tasks)

8.2 Backend Implementation

8.2.1 Search Users

`GET /users?search=username` (Autentkasi Diperlukan)

```
{
  "data": [
    {
      "id": "user-uuid",
      "username": "johndoe",
      "email": "john@example.com",
      "name": "John Doe"
    }
  ],
  "total": 1
}
```

8.2.2 Get User's Public Todos

`GET /users/:userId/todos` (Autentikasi Diperlukan)

Returns array of public todos only

8.2.3 Main List - Mixed Todos

`GET /todos` returns:

- User's own todos (private & public)
- Other users' public todos (read-only)

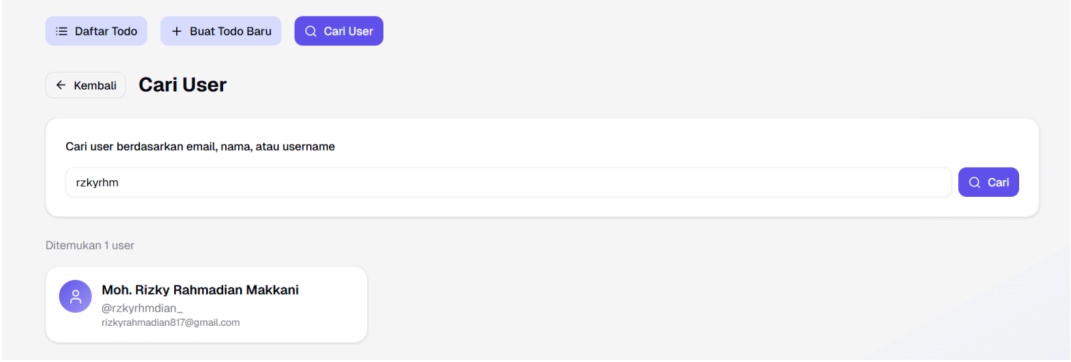
8.3 Frontend Implementation

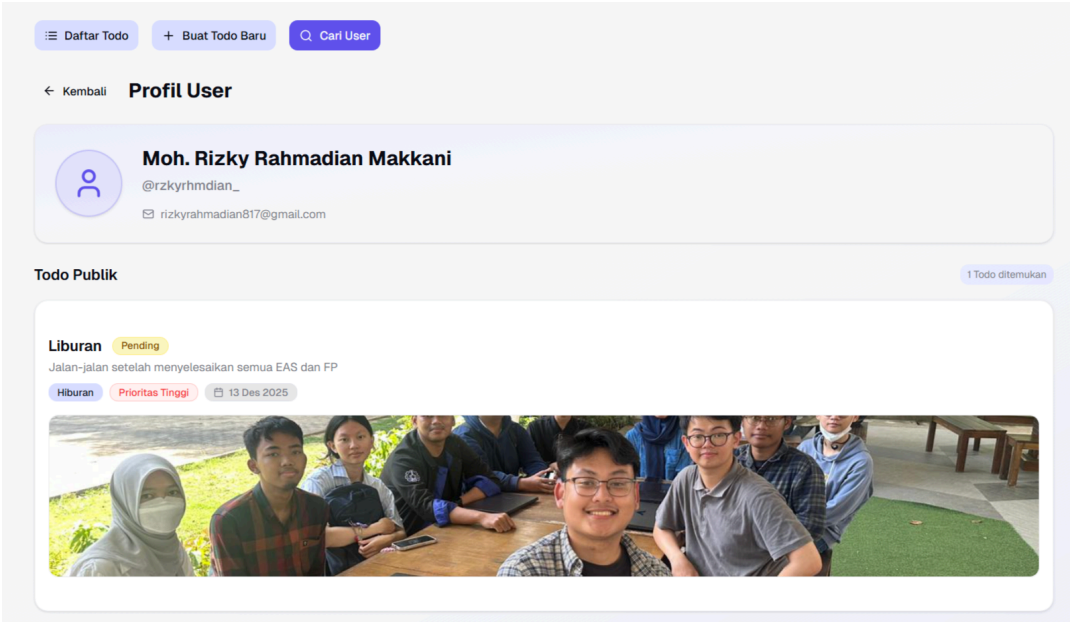
8.3.1 User Search Component

```
const handleUserSearch = async (query: string) => {
  const results = await apiClient.searchUsers(query)
  setUsers(results)
}
```

8.3.2 View Public Todos

```
const viewUserTodos = async (userId: string) => {
  const todos = await apiClient.getUserPublicTodos(userId)
  // Display in read-only mode
}
```





9 User Story 8: Email Reminders

Sebagai user, saya ingin menerima email reminder untuk deadline yang mendekat dan ringkasan task harian.

9.1 Fitur

- Automated deadline reminders (1 hari sebelum due date)
- Scheduled cron job execution
- Email dikirim jam 08:00 setiap hari (timezone Asia/Jakarta)
- Hanya kirim ke user dengan pending/upcoming tasks

9.2 Backend Implementation

9.2.1 Scheduler Service

```
@Injectable()
export class SchedulerService {
  constructor(private mailService: MailService, private prisma: PrismaService) {}

  @Cron('0 8 * * *')  // Every day at 08:00
  async sendDailySummary() {
    const users = await this.prisma.user.findMany()

    for (const user of users) {
      const pendingTodos = await this.prisma.todo.findMany({
        where: {
          userId: user.id,
          completed: false,
          dueDate: { gte: new Date() }
        }
      })

      if (pendingTodos.length > 0) {
        await this.mailService.sendDailySummary(user, pendingTodos)
      }
    }
  }
}
```

9.2.2 Email Template

Handlebars template (`reminder.hbs`):

```
Hi {{username}},

Ini adalah pengingat bahwa tugas berikut akan segera jatuh tempo:

Judul: {{ title }}
Tenggat Waktu: {{ dueDate }}

Segera selesaikan tugas Anda!

Salam,
Tim Todo App
```

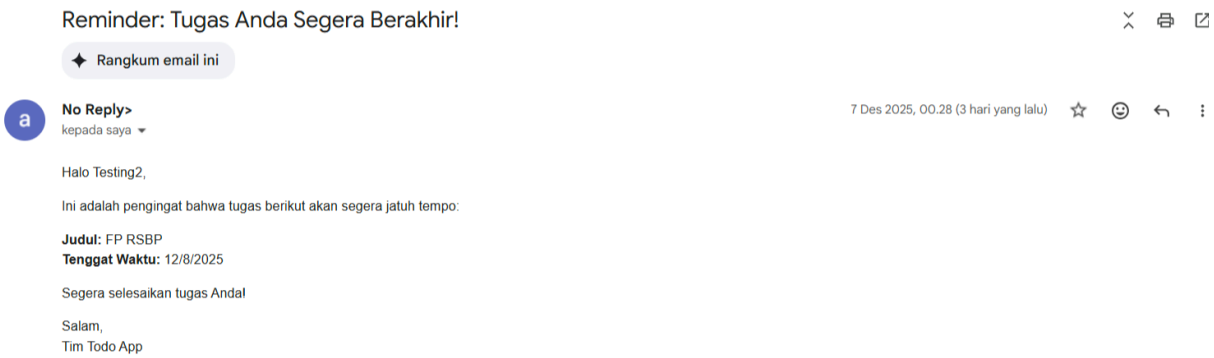
9.3 Configuration

Environment Variables:

```
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USER=your-email@gmail.com
MAIL_PASSWORD=your-app-password
MAIL_FROM=noreply@todoapp.com
```

9.4 Frontend Role

Frontend tidak ada interaksi khusus untuk US8. Email dikirim otomatis oleh backend scheduler.


Reminder: Tugas Anda Segera Berakhir!
Rangkum email ini
a
No Reply>
kepada saya
7 Des 2025, 00.28 (3 hari yang lalu)
Halo Testing2,
Ini adalah pengingat bahwa tugas berikut akan segera jatuh tempo:
Judul: FP RSBP
Tenggat Waktu: 12/8/2025
Segera selesaikan tugas Anda!
Salam,
Tim Todo App